

What is ReactJS?

1. Definition : ReactJS is an open-source JavaScript library used for building user interfaces (UI), especially for single-page web applications (SPA).

It was developed and maintained by Facebook.

2. Purpose of ReactJS : React is mainly used to create dynamic and interactive web interfaces where data changes frequently.

3. Key Features of ReactJS

- **Component-Based Architecture :** UI is divided into small reusable components.
- **Virtual DOM :** React uses a virtual DOM to update only changed elements, which improves performance.
- **Reusable Components :** Components can be reused in multiple parts of the application.
- **Fast Rendering :** Only necessary parts of the page are updated.

4. Working Concept (Simple Structure)

React Component



Virtual DOM



Compare with Real DOM



Update Only Changed Elements

5. Example

```
import React from "react";
function Welcome() {
  return <h1>Hello ReactJS</h1>; }
export default Welcome;
```

This component displays Hello ReactJS on the webpage.

Hello World Program

Steps to Create Hello World Program

Step 1: Create React Application

```
npx create-react-app myapp
```

Step 2: Move to Project Folder

```
cd myapp
```

Step 3: Start the Application

```
npm start
```

Hello World Program Code

Open App.js file inside the src folder and write the following code:

```
import React from "react";
function App() {
  return (
    <div>
      <h1>Hello World</h1>
    </div> ); }
export default App;
```

Output

Hello World

1. Install Node.js

- ReactJS requires Node.js and npm (Node Package Manager).
- Download and install Node.js from the official website.
- Verify installation using the command:

node -v

npm -v

2. Install Create React App : Create React App is a tool used to quickly set up a React project. **Command:** **npm install -g create-react-app**

3. Create a New React Application: Use the following command to create a React project.

npx create-react-app myapp

Here myapp is the project folder name.

4. Move to Project Directory

cd myapp

This command opens the React project folder.

5. Start the React Application

Run the following command to start the development server.

npm start

After running this command, the application will open in the browser.

6. Project Structure (Basic)

myapp

```
|— node_modules
|— public
|— src
|   |— App.js
|   |— index.js
|— package.json
```

- **src** – Contains React components and main code.
- **public** – Contains static files like HTML.
- **package.json** – Contains project dependencies.

Example

```
function App() {
  return <h1>Hello React</h1>;
}
```

This code displays Hello React when the application runs.

Create a first app

1. Install Node.js

Install Node.js and npm which are required to run React applications.

2. Create React Application

Use the following command to create a new React project:

npx create-react-app firstapp

3. Move to Project Folder

cd firstapp

4. Run the Application

Start the React application using:

npm start

Example Code (App.js)

```
import React from "react";
function App() {
  return (
    <div>
      <h1>My First React App</h1>
      <p>Welcome to ReactJS</p>
    </div> ); }
export default App;
```

Output: The browser displays “My First React App” on the webpage.

1. Definition : The ReactJS folder structure represents the organization of files and directories in a React application created using Create React App.

2. Basic Folder Structure

myapp

```

|
├── node_modules
├── public
│   └── index.html
├── src
│   ├── App.js
│   ├── index.js
│   └── App.css
├── package.json
└── README.md
  
```

4. Example : App.js file inside src folder contains the main React component that displays the UI of the application.

3. Explanation of Important Folders

- **node_modules**
Contains all installed project dependencies and libraries.
- **public**
Contains static files like index.html which is the main HTML file.
- **src**
Main folder where React components and application code are written.
- **package.json**
Contains project information, scripts, and dependencies.

Creating components

1. Definition : A component in ReactJS is a reusable piece of code used to create a part of the user interface (UI). Components help divide the UI into independent and reusable parts.

2. Types of Components

- **Functional Components**
- **Class Components**

3. Steps to Create a Component

1. Create a new JavaScript file inside the src folder.
2. Define the component using a function or class.
3. Export the component using export default.
4. Import and use the component in App.js.

4. Example (Functional Component)

File: Hello.js

```

import React from "react";
function Hello() {
  return <h1>Hello Component</h1>; }
export default Hello;
  
```

Using the component in App.js

```

import Hello from "./Hello";
function App() {
  return <Hello />; }
  
```

1. Definition : Components are the building blocks of a React application. They divide the user interface into small, reusable, and independent parts.

2. Types of Basic Components

1. Functional Component

A functional component is a simple JavaScript function that returns JSX to display UI elements.

Example:

```
function Welcome() {  
  return <h1>Hello Student</h1>; }  
}
```

2. Class Component

A class component is created using a JavaScript class that extends `React.Component`.

Example:

```
import React, { Component } from "react";  
class Welcome extends Component {  
  render() {  
    return <h1>Hello Student</h1>; } }  
}
```

3. Features of Components

- **Components are reusable.**
- **They help organize code properly.**
- **Each component works independently.**

4. Example in Application

```
function App() {  
  return (  
    <div>  
      <Welcome />  
    </div> ); }  
}
```

Here Welcome is used as a component inside the main App

class component

1. Definition : A Class Component in ReactJS is a component created using a JavaScript class that extends `React.Component`. It uses a `render()` method to return JSX that displays the user interface.

2. Characteristics of Class Components

- **Created using ES6 class syntax.**
- **Must extend `React.Component`.**
- **Uses `render()` method to display UI.**
- **Can use state and lifecycle methods.**

3. Syntax of Class Component

```
class ComponentName extends React.Component {  
  render() {  
    return (  
      <h1>Content</h1> ); } }  
}
```

4. Example

```
import React, { Component } from "react";  
class Welcome extends Component {  
  render() {  
    return (  
      <div>  
        <h1>Hello Student</h1>  
        <p>Welcome to ReactJS</p>  
      </div> ); } }  
export default Welcome;
```

5. Using Class Component in App

```
import Welcome from "../Welcome";  
function App() {  
  return (  
    <div>  
      <Welcome />  
    </div> ); }  
}
```

6. Output

The browser displays:

Hello Student

Welcome to ReactJS

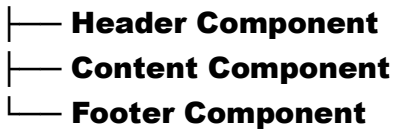
1. Definition : Nesting components means placing one component inside another component. It allows developers to build complex user interfaces by combining multiple components.

2. Purpose of Nesting Components

- Helps organize UI structure.
- Promotes code reusability.
- Makes application modular and easy to maintain.

3. Structure of Nested Components

App Component



Here the Header, Content, and Footer components are nested inside the App component.

4. Example

Header.js

```
import React from "react";
function Header() {
  return <h1>Welcome to Website</h1>;}
export default Header;
```

App.js

```
import React from "react";
import Header from "../Header";
function App() {
  return (
    <div> <Header />
    <p>This is the main page.</p>
    </div> ); }
export default App;
```

5. Output

Welcome to Website
This is the main page.

Functional component

Definition : A Functional Component is a simple JavaScript function used to create a React component. It returns JSX (JavaScript XML) to display the user interface.

2. Characteristics of Functional Components

- Written using JavaScript functions.
- Simpler and easier than class components.
- Used mainly for presentational UI.
- Can use React Hooks to manage state and lifecycle.

3. Syntax of Functional Component

```
function ComponentName() {
  return (
    <h1>Content</h1> ); }
```

4. Example

```
import React from "react";
function Welcome() {
  return ( <div>
    <h1>Hello Student</h1>
    <p>Welcome to ReactJS</p>
    </div> ); }
export default Welcome;
```

5. Using Functional Component in App

```
import Welcome from "../Welcome";
function App() {
  return <Welcome />; }
```

6. Output

The webpage displays:
Hello Student
Welcome to ReactJS

1. Definition

JSX (JavaScript XML) is a syntax extension of JavaScript used in ReactJS to describe the structure of the user interface. It allows developers to write HTML-like code inside JavaScript. **JSX makes React code simple, readable, and easy to maintain.**

2. Key Features of JSX

- **Allows HTML syntax inside JavaScript code.**
- **Makes UI code easier to understand.**
- **Supports JavaScript expressions using curly braces {}.**
- **Used to create React elements.**
- **Improves code readability and development speed.**

3. JSX Syntax Rules

- **JSX must have one parent element.**
- **Use className instead of class.**
- **All HTML tags must be closed.**
- **JavaScript expressions are written inside {}.**

4. JSX Program Example

```
import React from "react";
function App() {
  const name = "Student";
  return (
    <div>
      <h1>Hello {name}</h1>
      <p>Welcome to JSX in React</p>
    </div> ); }
export default App;
```

5. Explanation of Program

- **import React** – Imports the React library.
- **function App()** – Creates a React functional component.
- **const name = "Student"** – Declares a JavaScript variable.
- **{name}** – Displays the value of the variable inside JSX using curly braces.
- **<div>** – Acts as the main parent element.
- **return()** – Returns JSX code to display on the webpage.
- **export default App** – Exports the component to use in other files.

6. Output

The browser displays:

Hello Student

Welcome to JSX in React

ReactJS Props

Definition : Props (Properties) are used to pass data from one component to another, usually from parent component to child component.

Features

- Props are read-only.
- Used for communication between components.
- Passed as attributes in components.

Example

```
function Welcome(props) {  
  return <h1>Hello {props.name}</h1>; }  
function App() {  
  return <Welcome name="Student" />; }  
}
```

Output

Hello Student

React State

Definition : State is an object used to store dynamic data inside a component. The state can change during the lifecycle of the component.

Features

- State is mutable (can change).
- Used to control component behavior.
- Changes in state cause re-rendering of the component.

Example

```
import React, { Component } from "react";  
class Counter extends Component {  
  state = { count: 0 };  
  render() {  
    return <h1>{this.state.count}</h1>; } }  
}
```

Destructuring Props and State

Definition : Destructuring is a JavaScript feature used to extract values from props or state objects and use them directly.

Advantages

- Makes code shorter and cleaner.
- Improves readability.

Example

```
function Welcome({ name }) {  
  return <h1>Hello {name}</h1>; }  
State destructuring example:  
const { count } = this.state;
```

Definition : `setState()` is a method used in class components to update the state of a component.

When state changes, React re-renders the component automatically.

Example

```
class Counter extends React.Component {
  constructor() {
    super();
    this.state = { count: 0 };
  }
  increase() {
    this.setState({ count: this.state.count + 1 });
  }
  render() {
    return (
      <div>
        <h1>{this.state.count}</h1>
        <button onClick={() => this.increase()}>Increase</button>
      </div> );
  }
}
```

Methods as Props

Definition : In React, methods (functions) can be passed as props from a parent component to a child component.

This allows the child component to trigger functions in the parent component.

Example

```
function Child(props) {
  return <button onClick={props.show}>Click</button>;
}
function App() {
  function message() {
    alert("Hello from Parent");
  }
  return <Child show={message} />;
}
```